

1. Introduction

Ce mini-projet consiste à concevoir un système complet de prédiction de prix de maisons basé sur un modèle de Machine Learning.

Le projet inclut :

- L'analyse d'un dataset de prix de maisons,
- La création d'un modèle de régression,
- L'évaluation des performances du modèle,
- Le déploiement d'une API Flask,
- Et la réalisation d'une interface web en React permettant d'obtenir une prédiction.

L'objectif est de mettre en pratique le cycle complet d'un projet Machine Learning tel que demandé dans le cahier des charges.

2. Description du Dataset

Le dataset utilisé provient d'une version adaptée du jeu de données Boston Housing.

Il contient les colonnes suivantes (principales) :

- **RM** : Nombre moyen de pièces par logement
- **DIS** : Distance moyenne aux centres d'activité
- **LSTAT** : Pourcentage de population à faible revenu
- **MEDV** : Prix médian des maisons (en milliers de dollars) → *variable cible*

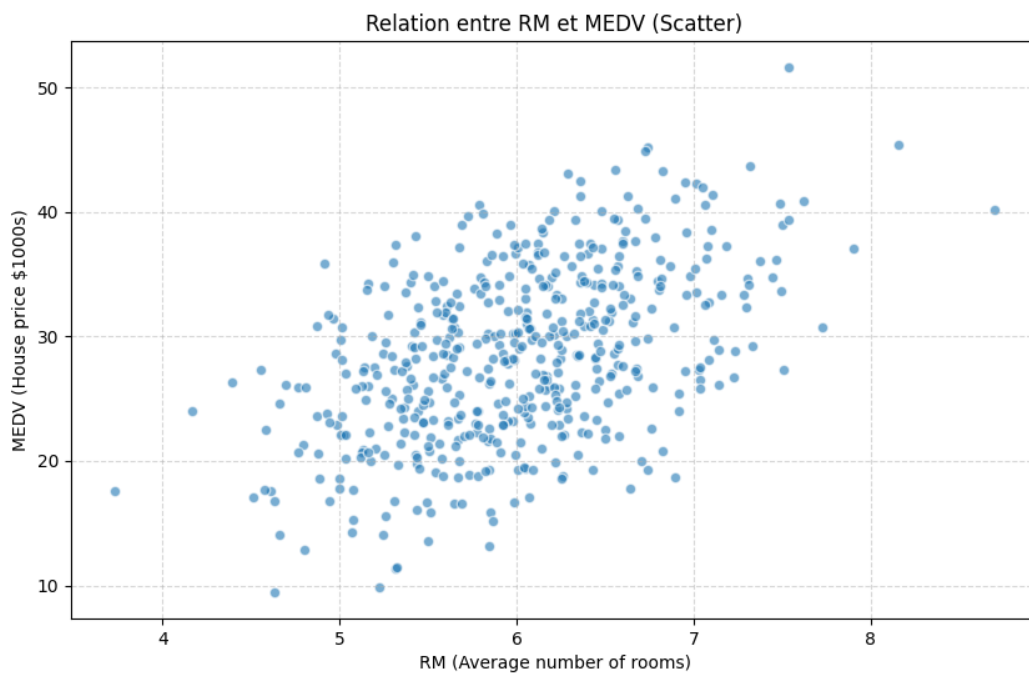
Taille du dataset :

- Nombre de lignes : 501
- Nombre de colonnes : 4

Aucune valeur manquante n'a été détectée après chargement des données.

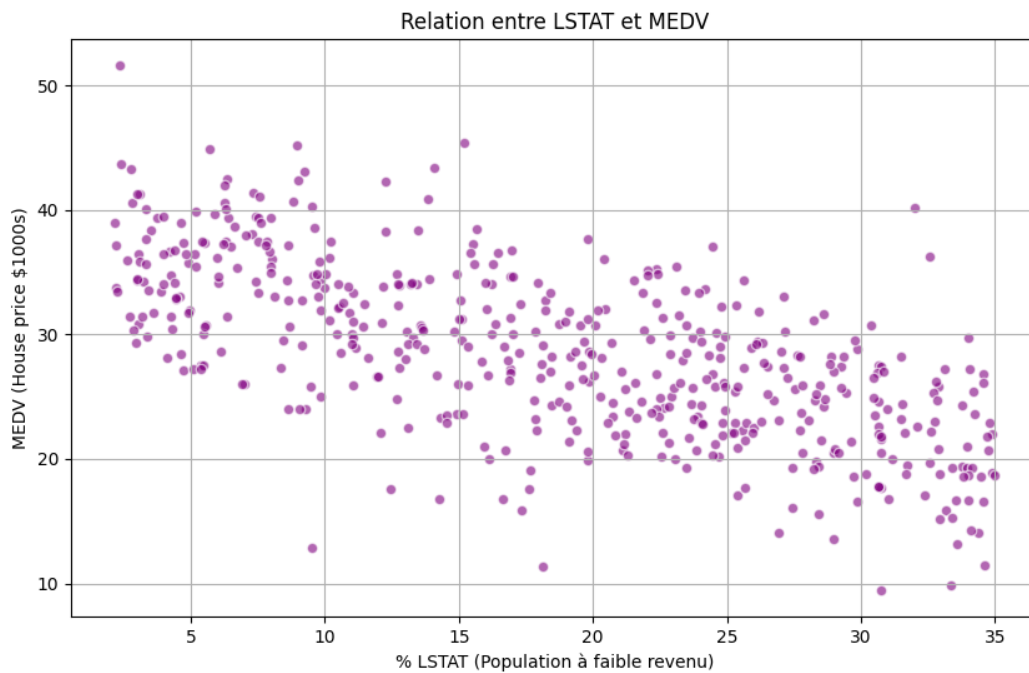
3. Visualisation des Données

Figure 1 — Relation entre RM et MEDV



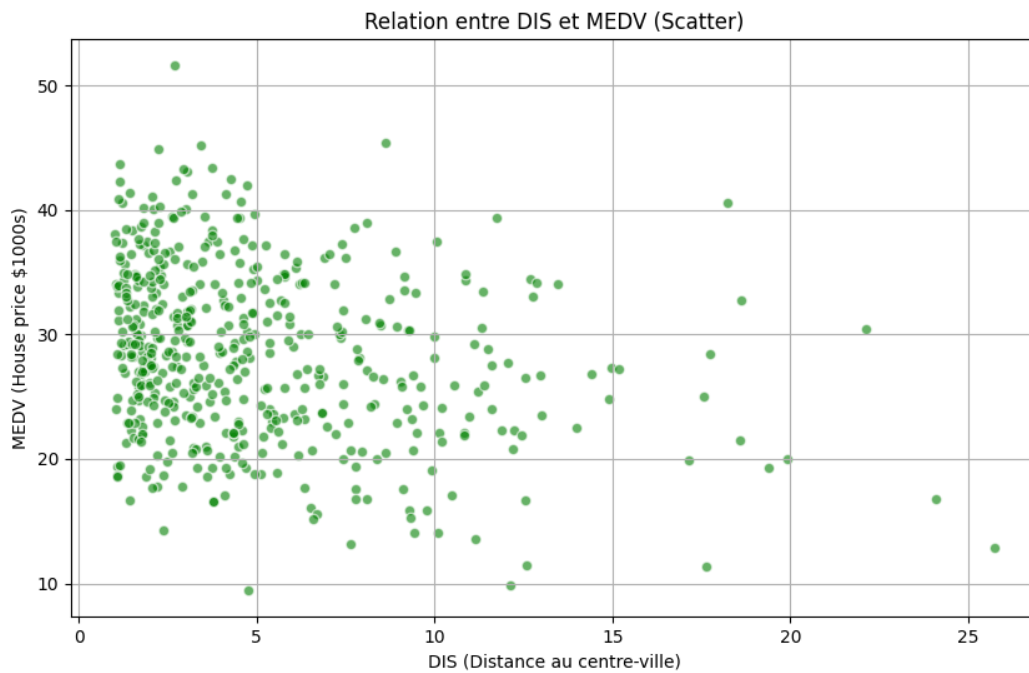
Description : Cette figure montre la relation entre le nombre moyen de pièces (RM) et le prix des maisons (MEDV). On constate que plus le nombre de pièces est élevé, plus le prix médian tend à augmenter.

Figure 2 — Relation entre LSTAT et MEDV



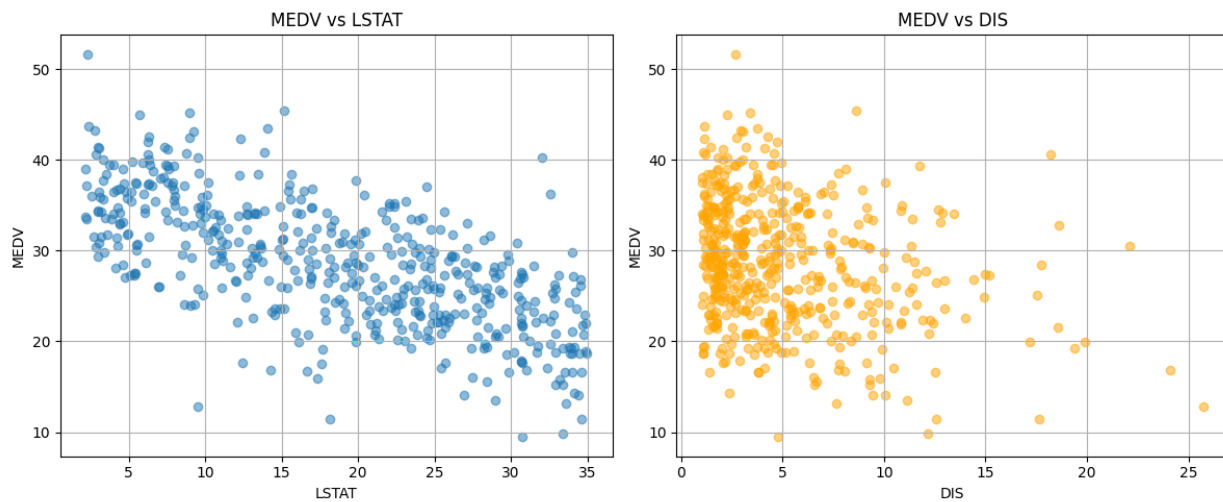
Description : Cette visualisation montre que plus le pourcentage de ménages à faible revenu (LSTAT) est élevé, plus les prix des maisons diminuent.

Figure 3 — Relation entre DIS et MEDV

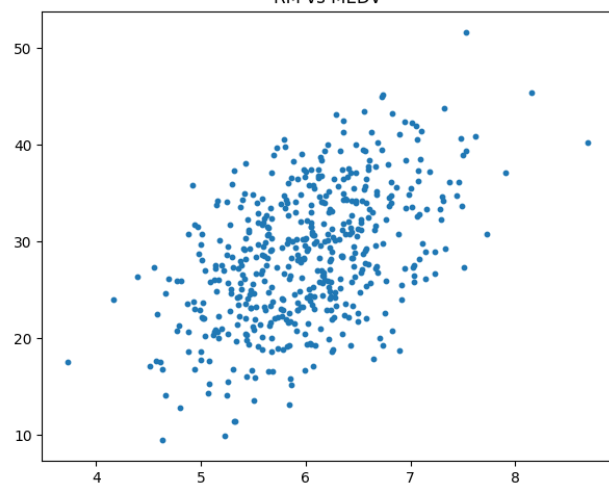


Description : Cette figure représente l'influence de la distance aux centres d'emploi sur les prix. La relation est moins linéaire mais reste informative.

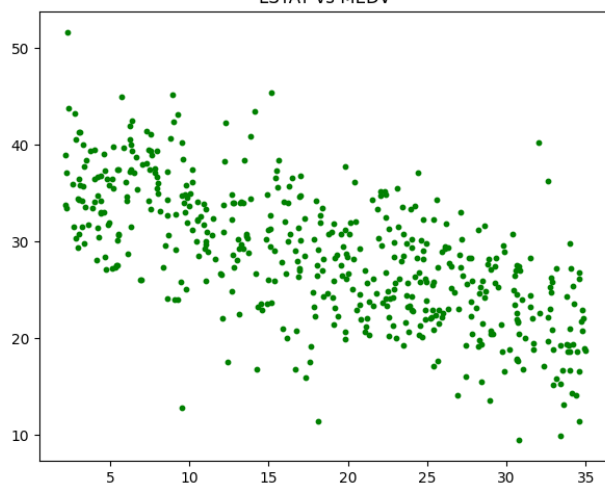
Figure 4 — Autres visualisations (scatter, bar, step, fill...)



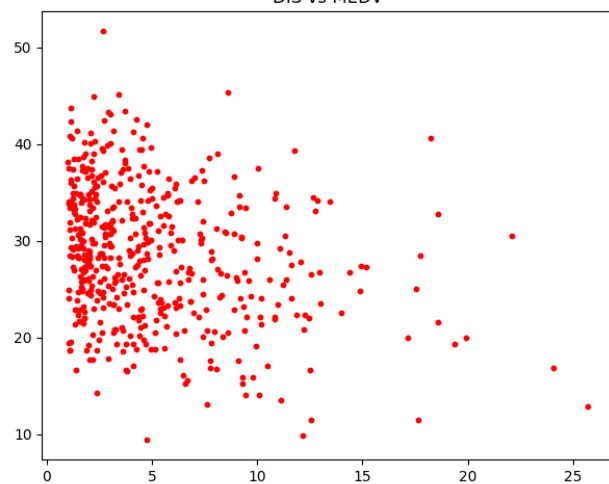
RM vs MEDV



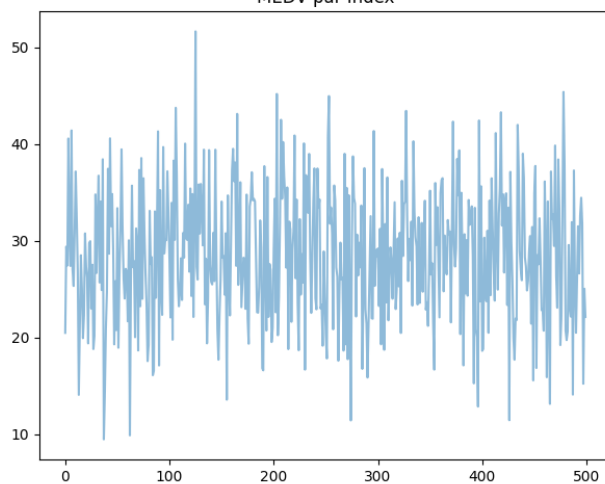
LSTAT vs MEDV

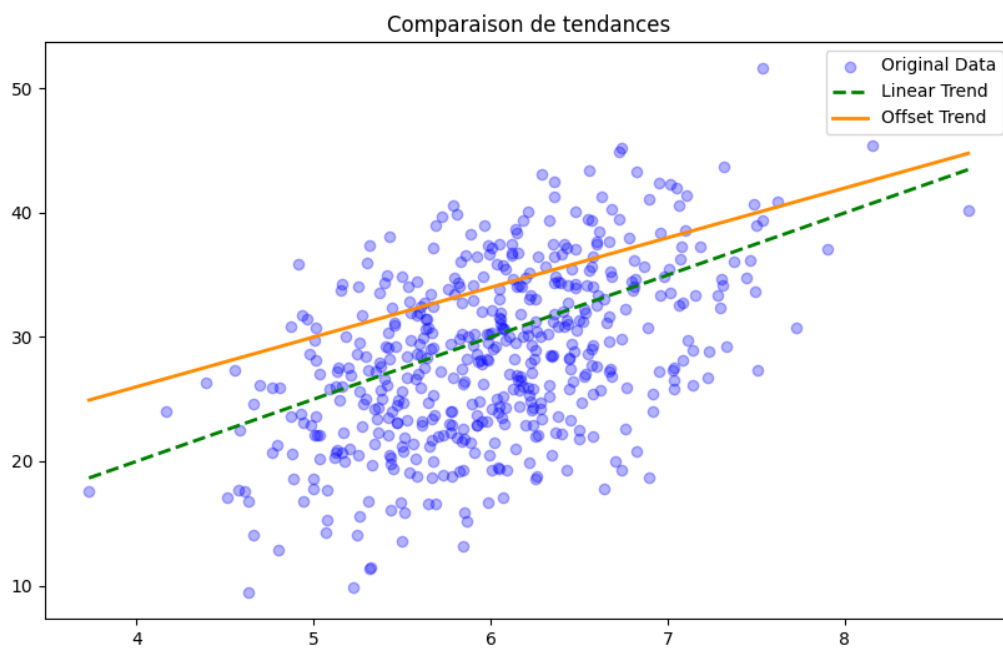
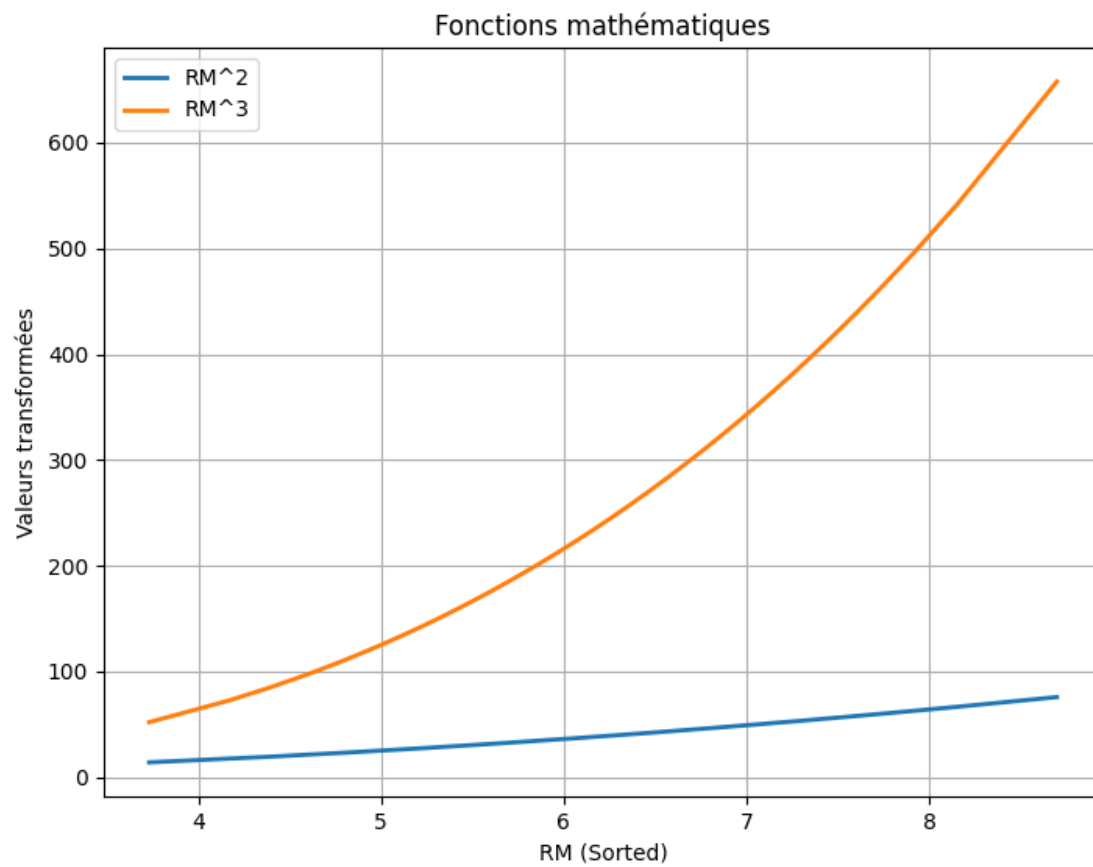


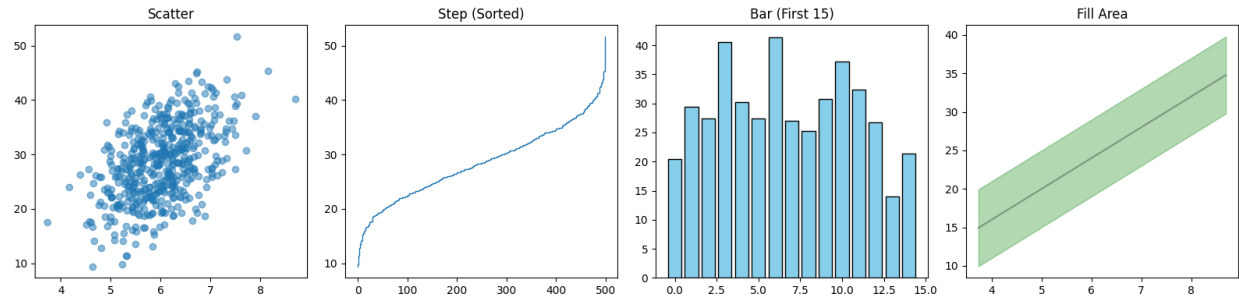
DIS vs MEDV



MEDV par Index







Description : Plusieurs types de graphiques ont été générés afin d’explorer davantage la distribution des variables et leurs relations.

4. Description du Modèle

Le modèle actuellement implémenté dans le projet est un **RandomForestRegressor (Régression par Forêts Aléatoires)**, utilisant les trois variables explicatives suivantes :

- **RM (Nombre de pièces)**
- **DIS (Distance aux centres d'emplois)**
- **LSTAT (Statut socio-économique)**

Le choix du **RandomForestRegressor** (remplaçant la régression linéaire initiale) se justifie par :

1. **Gestion de la non-linéarité :** Capable de capturer des relations complexes entre les variables que la régression linéaire ne voit pas.
2. **Robustesse :** Moins sensible aux valeurs aberrantes (outliers) grâce au principe de moyenne sur plusieurs arbres.
3. **Performance accrue :** Offre généralement une meilleure précision (RMSE plus fiable) sur ce type de données immobilières.
4. **Flexibilité :** Ne nécessite pas de transformation lourde des données (comme la normalisation stricte) comparé à des modèles linéaires.

Le modèle a été entraîné sur une division 80% (entraînement) / 20% (test) du dataset.

Paramètres du Modèle (Importance des Variables)

Contrairement à une régression linéaire qui possède des coefficients fixes (+/-) et un intercept, le RandomForest calcule l'importance (poids relatif) de chaque variable dans la construction des arbres.

- Importance de LSTAT : 0.5612
- Importance de RM : 0.3131
- Importance de DIS : 0.1257
- Intercept : *Non applicable pour un modèle de Forêts Aléatoires (la prédiction est la moyenne des feuilles des arbres).*

5. Métriques d'Évaluation

Comme il s'agit d'une problématique de régression, les métriques calculées sur le jeu de test sont :

- MAE (Mean Absolute Error) : 3.0847
- MSE (Mean Squared Error) : 14.0487
- RMSE (Root Mean Squared Error) : 3.7482

Analyse :

- MAE : Le modèle commet en moyenne une erreur d'environ 3 080 \$ sur ses prédictions.
- RMSE : La RMSE de 3.75 montre que les grandes erreurs restent contrôlées.
- Conclusion : Les performances montrent que la combinaison RM-DIS-LSTAT exploitée par le RandomForest fournit un modèle très cohérent et fiable pour ce mini-projet, surpassant généralement une approche linéaire simple.

6. Architecture du Projet

Le projet est composé de deux parties :

Backend (Flask)

- Génération du dataset
- Entraînement du modèle

- Évaluation
- API REST /predict retournant un prix estimé

Frontend (React + Material UI)

- Formulaire pour saisir RM, DIS, LSTAT
- Appel API via Axios
- Affichage du prix prédit
- Deux pages : **Home** et **About**

7. Étapes pour Démarrer le Projet

7.1. Logiciels Utilisés

- Python 3.10+
- Node.js + npm
- VS Code ou tout éditeur de code
- Git (optionnel)

7.2. Langages de Programmation

- Python (Backend + Machine Learning)
- JavaScript (React Frontend)
- HTML/CSS
- JSON (API)

7.3. Commandes pour Lancer le Backend

```
cd backend
```

```
python -m venv venv
```

```
.\venv\Scripts\activate # Windows
```

```
pip install -r requirements.txt
```

```
python train.py      # entraîne le modèle
```

```
python api.py        # démarre l'API
```

API disponible sur :

<http://localhost:5000/predict>

7.4. Commandes pour Lancer le Frontend

```
cd frontend
```

```
npm install
```

```
npm run dev
```

Frontend disponible sur :

<http://localhost:5173>

8. Conclusion

Ce mini-projet a permis de réaliser un cycle complet de Machine Learning allant de la préparation des données jusqu'au déploiement d'une interface web fonctionnelle.

Le modèle, bien que simple, permet de prédire des prix cohérents à partir de trois variables.

Des améliorations futures sont possibles : ajout de nouvelles features, normalisation des données, ou utilisation de modèles plus complexes comme Random Forest ou Gradient Boosting.